

An In-Context Surrogate Model for Computational Fluid Dynamics (Report 2)

Brodie Knight

Supervised by Max Zhu, James Rowbottom, Stefania Fresca, Pietro Liò

Abstract

This project presents a transformer-based surrogate model for Computational Fluid Dynamics, using an in-context learning approach to enable the model to handle different fluid equations via a secondary context encoder model. An adversarial discriminator network is also applied to improve output detail and physical accuracy. Two models are trained on hundreds of simulations generated from a numerical fluid solver, one trained over different bulk viscosity values, and the other trained over a whole set of differing parameters. The models are then tested with moderate success on out of sample datasets, for unseen fluid parameters, over a medium horizon autoregressive rollout.

1. Introduction

The first project report demonstrated that a patch based transformer architecture with generative adversarial network (GAN) based adversarial loss could successfully perform in-context learning with minimal artefacting. However, a core issue with such an approach is its lack of interpretability – a trait which is incredibly important for a CFD solver to possess. Secondly, it was determined that, while it appeared that the context encoder was learning to predict based on the provided context frames, in reality the model was relying on spurious patterns in the fluid state itself to infer the context information. This report attempts to address these issues, by presenting a new, more interpretable, two stage model, an improved context encoder, and a novel approach to generating the datasets to make it infeasible for the generative model to rely on spurious correlations.

Hence, this report attempts to answer two questions. The first is to determine to what extent the more interpretable two-stage transformer dynamics model and flow matching detail model architecture can act as a surrogate CFD solver for fluids. The second is to determine how well a separate context encoder can genuinely represent the fluid parameters, in such a way allows the model to generalise to unseen dynamics, and how to construct a training dataset to optimise for this.

The first stage dynamics model is very similar to the model presented in the first report, with the major changes being in how the training regime and context encoder. The other significant change is the removal of the adversarial loss – this model is now only trained on MSE loss, which intentionally generates some smoothing since the model is encouraged to collapse to the conditional mean. In this sense, this dynamics model attempts to learn and isolate the ‘deterministic’ part of the dynamics.

The second stage is applies an understanding of the physics of fluid systems. Since the fluid behaves as a fundamentally stochastic process from the perspective of our course grid, we require a generative step, which generates the fine details according to a distribution, rather than fitting to the mean. This detail stage uses a *flow matching* [12] model to generate detail which attempts to generate according to the same distribution as the training dataset, thus solving the blurring issue that exists if using mean squared error (MSE) loss. The model in the first report solves this by using an adversarial loss function, which is more efficient at training time, but is not directly trained to capture the correct distribution.

The more common approach when modelling physical systems with uncertainty is to train a single generative (diffusion or flow matching) model end-to-end [4, 9, 19, 11, 17]. However, given that the degree of uncertainty involved (per-step) in fluid modelling is often low, it seems reasonable to split the task into a deterministic part and a stochastic part, where each part can be handled by a model which is best designed for its respective task. This breakdown is effective for interpretability, as it splits the task of the solver into the two distinct components of the Reynolds decomposition: the deterministic ‘intra-cell’ model and the the non-deterministic ‘sub-pixel’ chaotic detail.

The approach of splitting the problem into a deterministic base model and a refiner with uncertainty was initially shown to be successful in weather modelling [14, 15]. Recent work as applied flow matching models to refine the output of numerical PDE solvers [5] and as a residual for machine learning surrogates [1, 3], which motivates the architectural choice of this report. Fan et al. [7] take a very similar approach to this project for the fluid prediction pipeline, but use a hybrid neural solver rather than a completely learned dynamics solver, and use a conditional diffusion model to capture the fine details, instead of a flow matching model (although they do not train for in-context learning).

The context encoder is also designed for maximum interpretability. Existing ‘in-context learning’ CFD surrogate models such as ICON[23], VICON[2] and Zebra[20] simply concatenate context frames onto the input of a monolithic generative model. This makes it very difficult to determine what the model is inferring from this context, or determine if the model is relying on spurious correlations. In fact, such spurious relationships were identified during training, and motivated a new training regime to encourage the model to actually learn to represent context. This is an important finding, as *no other in-context model appears to address this issue* in their training regime, suggesting that it is likely that these other models also suffer from this rather critical problem. However, since it is not straightforward to observe/interpret the encoded context in these models, it is explains why this issue has not been identified prior.

It should be noted that a wide variety of different architectures were experimented with during the second phase of this project. These include a transport map based advection architecture (to preserve detail) with a learned residual, a patch based architecture where tokens existed in a superposition between different points (to enable efficient allocation of compute across the domain), and a hierarchical, self-similar patch based model (to replicate the self similar nature of fluid systems at various scales). However, as often seems to be the case with deep learning, especially when vast quantities of data are available, the most effective architectures are generally surprisingly simple. The general consensus is that the more architectural complexity is added, the more obstacles are introduced to inhibit the model to deep learn the complex structures and patterns of the data. This pattern was definitely observed during the course of this project. However, since some of the findings from these ultimately abandoned approaches were worth documenting, key details are included in the appendix.

2. Model Architecture

The model presented in the first report was successful in its ability to represent detail and model fluid flows. It used a standard image transformer with additional ‘free’ tokens to model fluids, but it struggled specifically when modelling high frequency detail for a more general distribution of fluids, especially over larger timesteps. This can be explained by the adversarial loss approach. While the adversarial loss was effective at sharpening the more diffusive output of the MSE loss trained model, the resulting output of the model was not physically representative. This is because a generative adversarial network (GAN) will tend to resolve the inconsistencies that are most obvious to the discriminator model, and miss ones which are difficult for the discriminator to pick up given its architecture. As such, the detail that the discriminator fixes are biased towards whatever the discriminator architecture is able to discriminate most easily.

The approach presented in this report attempts to resolve this by training two separate models. The *dynamics stage* (stage 1) is the familiar patch image transformer architecture from report 1, without the discriminator network or adversarial loss. The *detail stage* (stage 2) is comprised of a flow matching model, which takes the diffuse output of the dynamics stage and adds detail according to a learned distribution of the chaotic detail. As such, this approach aligns in essence with the framing of the Reynolds decomposition: the dynamic stage represents the deterministic, inter-cell information, and detail stage represents the distribution of the chaos which cannot be modelled due to the coarseness of the data.

2.1. Context Encoder

Both dynamics and detail stages depend on the tokens produced as output from the context encoder to encode the context that their output is conditioned on. The context encoder, as depicted in figure 1, accepts a set of $T = 5$ consecutive frames (separated by a constant timestep) and encodes 16×16 cell patches for each frame. Each patch can be expressed as

$$z_{t,ij} = W_{\text{ctx}} \text{vec}([f_t \parallel \Delta f_t \parallel m]_{\mathcal{P}_{ij}}) + e_{ij}^{\text{spat}} + e_t^{\text{temp}}, \quad \Delta f_t = \begin{cases} f_t - f_{t-1}, & t > 0, \\ \mathbf{0}, & t = 0, \end{cases}$$

where $\mathcal{P}_{ij}$ denotes the (i, j) -th non-overlapping 16×16 patch, $\parallel$ is channel-wise concatenation, $f_t \in \mathbb{R}^{4 \times 256 \times 256}$ the t -th context frame ($t = 0, \dots, T-1$, sampled at temporal stride s), m the binary geometry mask (set to 1 if the cell contains the fluid, and 0 if not), and $W_{\text{ctx}} \in \mathbb{R}^{d_{\text{ctx}} \times (16 \cdot 16 \cdot 9)}$ a linear projection shared across all patches and frames. $e_{ij}^{\text{spat}}, e_t^{\text{temp}} \in \mathbb{R}^{d_{\text{ctx}}}$ are learned absolute spatial and temporal position embeddings, giving $T \cdot 16^2$ tokens with $d_{\text{ctx}} = 384$.

For embedding context, the inclusion of the frame delta, Δf_t , trivially provides intra-frame dynamics. The version of the context encoder presented in report 1 embedded did not directly include the frame deltas, which forced the context encoder to learn how to compare information between frames. Since the gradient signal into the context encoder is relatively weak, this additional step is prohibitively difficult for the context encoder to learn. This finding was motivated by an empirical analysis of the context encodings in the original model, which actually showed that the context encoder was learning essentially nothing, and any apparent in-context ability of that model could be attributed to overfitting patterns in the fluid states.

The inter-temporal patches each have fixed spatial and temporal positional embeddings applied (e_{ij}^{spat} and e_t^{temp} respectively), and flow through 4 transformer layers with global self-attention across the patch tokens and cross attention between the patch tokens and the 64 query tokens of

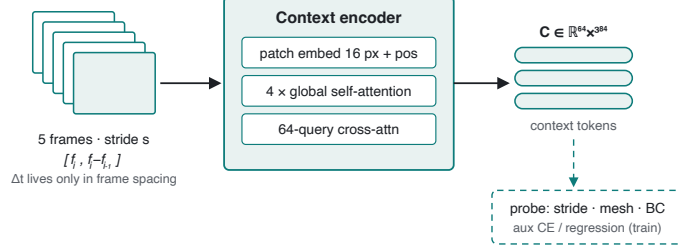


Figure 1: Context Encoder

size $d_{\text{ctx}} = 384$. These query tokens exit the model as the encoded context, for the generative step to condition on.

The frames that are fed to the context encoder with a random stride of $s \in \{1, 2, 4\}$ (meaning that subsequent states fed into the model are s frames apart). This means that the context encoder will now encode variable evolution timesteps for the generative stages. Since all the generated data used the same solver timestep, $\Delta t = 0.1$, the Δt values that the model can represent are limited to $\Delta t \in \{0.1, 0.2, 0.4\}$, but success for this reduced set of Δt should generalise to an arbitrary range of small values of Δt .

2.2. Dynamics Stage – Context Encoded Patch Transformer

The first stage of the generative model, the dynamics stage, is the backbone of the solver, responsible for solving the vast majority of the physics. This dynamics model (unlike the version presented in the first report) predicts the *change in fluid state* instead of the absolute fluid state. This change was made because as the training set grew, it became clear that the model no longer had the capacity to represent sufficiently diverse distribution of different fluid states via its patch embeddings. Since the fluid state deltas lie on a far smaller manifold than the absolute fluid states, it is much more manageable for the model to accurately output fluid deltas, because the latent space is able to allocate its capacity to a far smaller range of potential outputs.

As discussed in the first report, this introduces the issue around autoregressive stability, because a predicted delta that is even slightly off can very quickly result in the fluid states degenerating into chaos. This was addressed by training over multiple timesteps in one iteration, and utilising back-propagation through time (BPTT) to teach the model how to ‘self-correct’ if its output is slightly misaligned. Training over multiple timesteps was also very effective for eliminating patch boundary artefacts, in the absence of the adversarial loss term.

2.2.1. Encoder

This model uses patch embeddings, as in the context encoder. However, unlike the context encoder, the dynamics model takes a single, static, fluid state, and contains no data from past fluid states. The patch embeddings for the dynamics model can be given as:

$$h_{ij} = W_{\text{dyn}} \text{vec}([x_t \parallel m]_{\mathcal{P}_{ij}}), \quad \mathcal{T} = (h_{ij})_{i,j=1}^{16} \cup (g_k)_{k=1}^{16},$$

where $\mathcal{P}_{ij}$ denotes the (i, j) -th non-overlapping 16×16 patch, $x_t \in \mathbb{R}^{4 \times 256 \times 256}$ is the current fluid state (with the cells outside of the fluid mask set to zero), m is the binary geometry mask, and

$W_{\text{dyn}} \in \mathbb{R}^{d \times (16 \cdot 16 \cdot 5)}$ is a linear projection shared across patches. The $g_k \in \mathbb{R}^d$ are 16 learned global capacity tokens appended to the sequence, giving $16^2 + 16$ tokens with $d = 384$. No additive positional embeddings are used; instead the spatial position of the patch tokens are expressed via two-dimensional rotary positional embedding (RoPE) [21].

2.2.2. Transformer

The transformer layer in this model are shown in figure 4. Each layer starts with a global self attention block, where the 2D RoPE embeddings are applied to the query and key matrices

$$\langle R_{ij} q, R_{i'j'} k \rangle = \langle R_{i-i', j-j'} q, k \rangle, \quad R_{ij} = \text{RoPE}_{2D}(i, j), \quad R \equiv I \text{ for } g_k,$$

so that attention logits depend on patch positions only through their relative offset $(i - i', j - j')$; the global tokens receive the identity rotation and so do not carry positional information.

The tokens then pass through a cross-attention sub-layer, enabling the $16^2 + 16$ generative tokens to attend to the 64 context tokens. This enables the information held within the context to inform the dynamics of the model. Finally, the generative tokens pass through a feed forward sub-layer, with an expansion ratio of 4 : 1.

2.2.3. Decoder

The decoder block starts with a feed forward network, which decodes the token into 24×24 overlapping patches. Tent weightings are applied given average weighted values across the overlapping regions, with weights in the overlapping areas determined as a linear function from zero to one of distance from the overlap threshold, given as

$$\hat{y}(q) = \frac{\sum_{i \in \mathcal{N}(q)} w_i(q) \hat{y}_i(q)}{\sum_{i \in \mathcal{N}(q)} w_i(q)}$$

where $\mathcal{N}(q)$ is the set of patches which contain point q , $w_i(q)$ is the per cell weight for patch i , and $\hat{y}_i(q)$ is the contribution from patch i .

As in the model presented in the first report, a skip encoder is used to pull sharp details from the input directly to the output. This makes it much easier for the model to handle fine details, because the detail does not need to survive 12 transformer layers, as it would have to if there were no skip encoder. Despite predicting the delta instead of the absolute fluid state, the residual connection is still very useful, because the delta is a function of the fine detail of the absolute state. So even though the actual values of the delta are very different, the geometric features are shared.

At this point, the cell values that are outside the fluid domain (inside collision geometry) may be set to arbitrary values by the model output. Hence, a masking step is applied to zero all cells outside the fluid domain (if this was not performed, it would cause strange artefacts around the collider boundaries because the convolutional layers would overlap with the cells that should have been masked but instead contain junk).

The skip encoder uses a thin 3×3 convolutional layer with 32 channels, with output $\mathbf{x}_{\text{skip},t}$. This is then fed, along with the 4 output channels from the patch decoded output, into a second, 64 channel 3×3 convolutional layer which decodes into an additive correction term, which is applied to the patch decoded output, giving the final model output.

$$\hat{\mathbf{x}}_{t+1} = \hat{\mathbf{y}}_{t+1} + \text{post_conv}([\hat{\mathbf{y}}_{t+1}, \|\mathbf{x}_{\text{skip},t}\])$$

2.2.4. Loss

The loss contribution for each iteration is normalised by scaling according to the inverse of the hypothetical loss that would be the result of simply persisting the previous fluid state. In order to stabilise over autoregressive rollouts, the model is trained with back-propagation through time (BPTT) with each training rollout consisting of $H = 4$ steps. Hence the loss function of the dynamics model is given as

$$\mathcal{L} = \frac{1}{H} \sum_{k=1}^H \frac{\ell(\hat{\mathbf{x}}_{t+k}, \mathbf{x}_{t+k})}{\max(\ell(\mathbf{x}_t, \mathbf{x}_{t+k}), \epsilon)}, \quad \hat{\mathbf{x}}_{t+k} = \Phi_\theta(\hat{\mathbf{x}}_{t+(k-1)}, \mathbf{z})$$

with

$$\ell(\hat{y}, y) = \frac{\sum_q m(q) \|\hat{y}(q) - y(q)\|_2^2}{C \sum_q m(q)} + \lambda_1 \frac{\sum_q m(q) \|\hat{y}(q) - y(q)\|_1}{C \sum_q m(q)}$$

where q indexes the 256^2 spatial cell positions, $m(q) \in \{0, 1\}$ is the fluid mask ($m = 1$ on fluid cells, 0 inside solid geometry), $C = 4$ is the number of fluid channels and $\lambda_1 = 10^{-2}$ weights the L_1 term. Φ_θ denotes one autoregressive step of the model conditioned on the context tokens $\mathbf{z}$. The predictions are then fed back through the model, so that gradients propagate through all H steps. Each term is divided by the persistence baseline (the loss if the fluid is not changed at all) $\ell(x_t, x_{t+k})$, so that every iteration contributes gradient of comparable magnitude. The term $\epsilon = 5 \times 10^{-3}$ guards against the case where the fluid is unmoving.

Without normalisation, the model would prioritise parameter updates for more difficult predictions where more has changed. Hence, if we had a sample in the training corpus where very little happens, there would be much less pressure to learn to adapt to this scenario, because the gradient updates would be dominated by samples where prediction is harder and the error is naturally higher. This means that the model would not learn to deal with ‘easy’ scenarios as effectively as it should.

2.3. Detail Stage – Conditional Flow Matching Refiner

The detail stage attempts to predict stochastic detail that the dynamics model cannot capture, since it is trained under $L_1 + L_2$ loss. A deterministic model such as this will always produce a smoothed output that is correct on average but this output will not converge to the true distribution of fluid states.

Flow matching models, introduced by Lipman et al. [12], are a type of continuous normalising flow (CNF) approach. They learn a velocity field that transports a multi-variate Gaussian, $\mathbf{d}_0 \sim \mathcal{N}(0, I)$ to a target distribution, $p(\mathbf{d}_1)$ by integrating an ODE described by the vector field $u_\theta(\tau, \mathbf{d} | c)$. As such, they predict a distribution which is sampled by randomly sampling a point on the starting Gaussian $\mathbf{d}_0$, and transporting to the a point in the destination domain, such that $p(\mathbf{d}_1 | c)$ matches the true conditional distribution.

For the implementation used in this project (figure 2), the training target of the detail model is the residual of the output of the dynamics stage against the true ground truth fluid state. Hence:

$$\mathbf{d}_1 = \mathbf{x}_{t+1} - \hat{\mathbf{x}}_{t+1}$$

which means that the model learns the distribution of this residual. The distribution is conditioned on

$$c = (\mathbf{x}_t, \hat{\mathbf{x}}_{t+1}, \mathbf{z}, \mathbf{m})$$

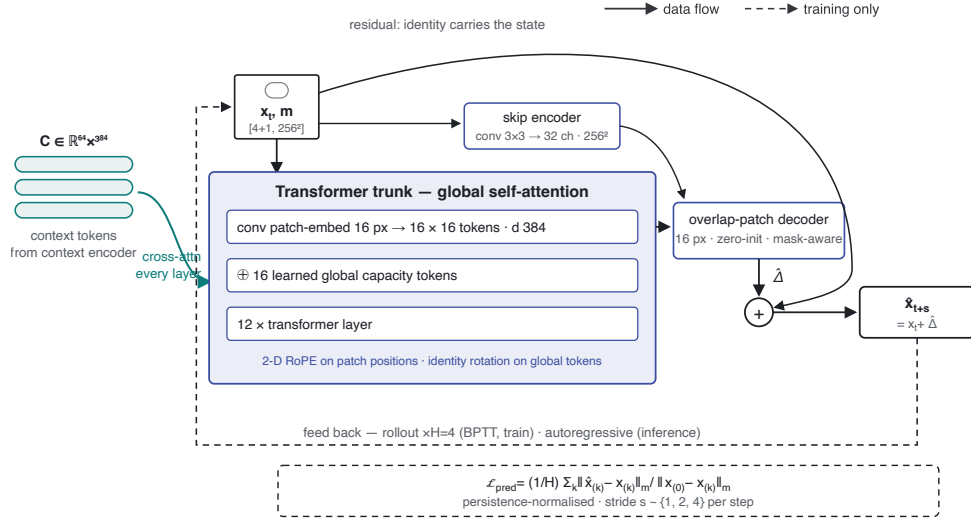


Figure 2: Patch Transformer Architecture

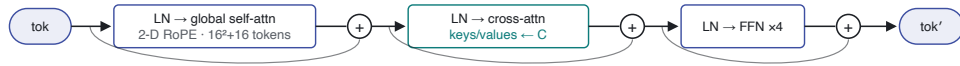


Figure 3: Single Patch Transformer Layer

where $\mathbf{x}_t$ is the (true or model generated) previous fluid state, $\hat{\mathbf{x}}_{t+1}$ is output of the dynamics model, $\mathbf{z}$ are the embedded context tokens and $\mathbf{m}$ is the fluid mask. The idea is that, because most of the difficult dynamics is taken care of by the dynamics model, the flow matching model is able to focus on what it is best at doing: learning the distribution of the residual.

At training time, a realisation from the Gaussian distribution $\mathbf{d}_0$ is drawn and matched with a draw from the training corpus, $\mathbf{d}_1$. Points drawn from the starting Gaussian are at $\tau = 0$ and points at the target distribution (i.e. the training data) sit at $\tau = 1$. The flow matching models draws a straight line segment from this $\mathbf{d}_0$ to $\mathbf{d}_1$, through $\tau \in [0, 1]$.

The model is trained by drawing random $\tau \sim \mathcal{U}(0, 1)$, $\mathbf{d}_0 \sim \mathcal{U}(0, I)$ and training the model on the loss function

$$\mathcal{L}_{\text{FM}} = \mathbb{E}_{\tau, \mathbf{d}_0, c} \left\| u_{\theta}(\tau, \mathbf{d}_{\tau} | c) - (\mathbf{d}_1 - \mathbf{d}_0) \right\|_{\mathbf{m}}^2, \quad \mathbf{d}_{\tau} = \tau \mathbf{d}_1 + (1 - \tau) \mathbf{d}_0,$$

so that the velocity field at any point tends to the average of $(\mathbf{d}_1 - \mathbf{d}_0)$ over the training data.

Different methods of learning $u_{\theta}(\tau, \mathbf{d} | c)$ can be chosen depending on the nature of the distribution to be learned. In this case, a U-Net model was chosen, as this is what is most commonly used in image generation flow matching models. The approach used in this project closely mirrors the U-Nets are a neural network architecture which progressively embed in a hierarchical manner, from fine to course (encoder size), then from course to fine (decoder side), in an inverted ‘U’ shape. The key property of U-Nets is that they have skip embeddings which carry data across from the encoder side to the corresponding decoder level, meaning that high frequency details are not lost as data gets compressed through the bottleneck of the courser layers.

A 13 dimensional concatenated frame, $[\mathbf{d}_{\tau} \| \mathbf{x}_t \| \hat{\mathbf{x}}_{t+s} \| \mathbf{m}] \in \mathbb{R}^{13 \times 256 \times 256}$, is run through the U-Net. The output head on the decoder then takes the 64 channel output and reduces it to the output vector size of 4×256^2 . Thus, the U-Net progresses through layers of size

$$13 \times 256^2 \rightarrow (64 \times 256^2 \rightarrow 128 \times 128^2 \rightarrow 192 \times 64^2 \rightarrow 128 \times 128^2 \rightarrow 64 \times 256^2) \rightarrow 4 \times 256^2$$

where each layer contains a FiLM sub-layer[16], which takes an embedded vector over all the context tokens and outputs both scale and shift offsets for the convolutional layer. This layer architecture, introduced by Dhariwal and Nichol [6], means that the context, $\mathbf{z}$, is able to amplify/augment the convolutional layers depending on the context it embeds. The network is zero-initialised, which means that the conditioning initially does nothing, and can slowly be learned as the model starts to rely on the context to reduce loss.

3. Model Training

Training the generative model to take the context into consideration is challenging, because it turns out that it is much easier for the model to ‘cheat’, by avoiding relying on the context. A general rule of machine learning models is that they will take the easiest path when given training data with specific correlations, which might result in the model relying on spurious assumptions about what constitutes an appropriate signal for the fluid parameters.

This provides further strong motivation for using this separate context encoder, as it creates a verifiable bottleneck through which to test how the fluid responds to different context encodings. Standard in-context surrogate models include everything in a single monolithic model, which makes it very difficult to determine if the in-context learning it exhibits is genuinely extracting the correct features, or if it just relies on spurious patterns within the training data. This is

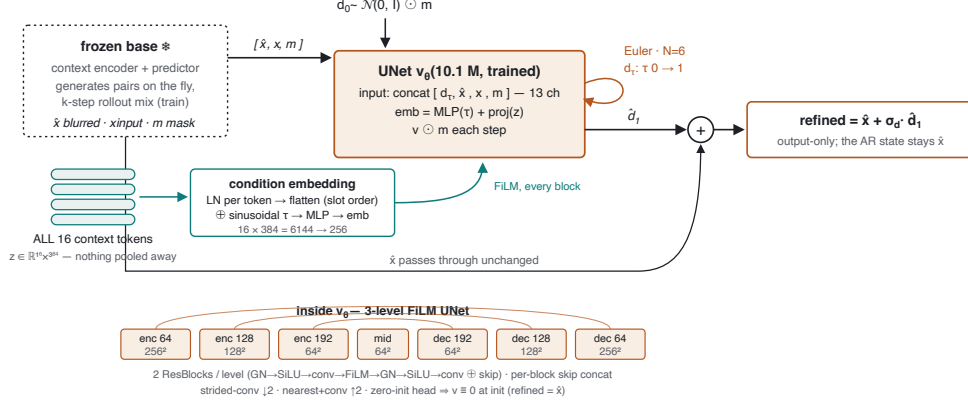


Figure 4: Conditional Flow Matching Architecture

significant because in order to understand how the solver behaves when presented with out of sample data, we must possess a way to observe how the model is interpreting the input that it is given during model inference.

When using a context encoder based approach, it provides a powerful tool which can offer such an understanding. Since the context tokens are explicitly provided, and are fixed for a given rollout, it is possible to inspect the values of the context tokens encoded by the context encoder, and determine whether they actually embed the correct data about the fluid dynamics of the model.

3.1. Training Data

A total of approximately 600,000 fluid frames, over 1,700 fluid trajectories were used to train the model, each generated using the PyTorch based fluid solver developed by Zhu [24]. These data differ substantially to those of the training corpus described in the first project report; the new data vary across collider geometry, fluid flow direction, boundary conditions and physical fluid parameters, and are generated in such a way to increase the pressure on the context encoder to learn context.

To start with, 24 different geometries are randomly initialised, by positioning a set of $n \in \{1, 2, 3, 4\}$ ellipses, with equal probability within the fluid domain. If multiple ellipses collide then the positions are resampled until there are no collisions.

3.1.1. Trajectory Samples

The *context* describes the physical fluid parameters and boundary conditions. Notably, these cannot be determined by observing a single fluid state – instead they parametrise the evolution operator, and thus can only be determined by the context encoder through analysis of how the

fluid changes over time. The values are distributed according to:

$$\begin{aligned}
\text{visc_model} &\sim \mathcal{U}\{\text{Newtonian}, \text{PowerLaw}, \text{Carreau}, \text{HerschelBulkley}\}, \\
n_{\text{visc}} &\sim \mathcal{U}(0.3, 1.8), \quad f_{\text{min}} \sim \mathcal{U}(0.1, 0.6), \\
\gamma_{\text{scale}} &\sim \text{LogU}(2.0, 40.0), \quad \mu \sim \text{LogU}(3 \times 10^{-4}, 6 \times 10^{-2}), \\
\mu_{\text{bulk}} &\sim \text{LogU}(10^{-3}, 3 \times 10^{-1}), \quad \kappa \sim \text{LogU}(5 \times 10^{-7}, 5 \times 10^{-3}), \\
C_v &\sim \mathcal{U}(1.5, 3.0), \quad \gamma \sim \mathcal{U}(1.1, 1.67), \\
\Delta t &\sim \mathcal{U}(0.01, 2.0).
\end{aligned}$$

The transport coefficients are drawn log-uniformly because they span two to four orders of magnitude, and a uniform draw would place almost all systems in the high-viscosity, high-conductivity regime. The viscosity model is categorical, so a quarter of all systems are non-Newtonian, and n_{visc} spans both shear-thinning ($n < 1$) and shear-thickening ($n > 1$) behaviour. The solver timestep, Δt , is also chosen randomly, so that the model can learn to evolve the fluid states by varying timesteps.

The set of *initial conditions* (ICs) holds initial fluid states for each trajectory, and are distributed according to:

$$\rho_{\infty} \sim \mathcal{U}(0.5, 2.0), \quad T_{\infty} \sim \mathcal{U}(80, 140), \quad v_{\infty} \sim \mathcal{U}(2, 7), \quad \alpha \sim \mathcal{U}(0^{\circ}, 360^{\circ}).$$

For each of the 24 geometries, 12 different starting contexts, and 6 different starting ICs are generated. A trajectory is then solved for every combination of context and IC draw. Hence, there are $12 \times 6 = 72$ trajectories per geometry and $24 \times 72 = 1,728$ trajectories in total.

Each of the 864 simulations runs for 12,000 solver iterations, with a frame saved every $\Delta t_{\text{save}} = 0.01$ of simulation time. Frames are stored as cell-level primitives (ρ, u, v, p) on the unstructured mesh and rendered to a 256×256 pixel grid at load time. The first 0.7 seconds of rollout so that the model can focus on learning more stable flows.

3.1.2. Context Switching

In order to ensure that the context encoder is used by the model, it is necessary to design the training data corpus in such a way that prevents the model from learning shortcuts through spurious correlations. This is done by switching the context for trajectories after a fixed number of timesteps.

Figure 5 shows how each trajectory is split into segments, within which all fluid parameters remain constant. The length of each segment was chosen to be 30 frames, strike a balance between preventing spurious learning and allowing the model to observe realistic rollout trajectories. Thus, every trajectory that is generated has its context (including the timestep, Δt) resampled after 30 frames of rollout by the numerical solver. Note that the boundary conditions do not change along segments, but stay constant over an entire trajectory, otherwise this would result in the model being trained on dynamics that are very unusual.

Context switching that the model cannot rely on the current fluid state to infer what the fluid parameters may be, because we could have reached the current state from a different set of fluid parameters to the ones that the model is using to predict the next state.

3.2. Context Encoder Probe

In order to improve training efficiency, a probe is attached to the output of the context encoder. This allows an extra training gradient to nudge the context encoder to capture the relevant details,

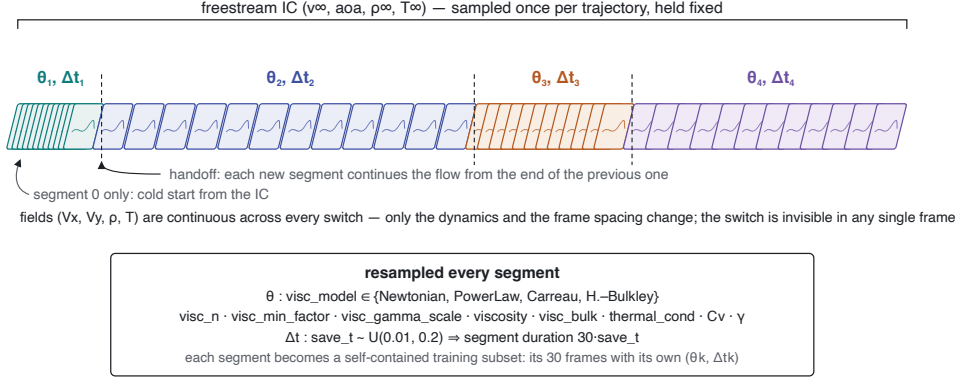


Figure 5: Training Trajectory with Context Switching

so that the context encoder learns more quickly, and the dynamics model can begin to utilise this context earlier.

The probe takes a simple component-wise mean over all 64 context tokens, passes this through a small linear layer plus activation function, and runs three prediction heads over the output. These three probes predict the stride length s , the mesh topology, and the context parameters. The stride and mesh topology heads are trained with cross-entropy loss, and the context parameter head is trained with MSE loss.

Analysis of the gradient magnitudes shows that 70% of the gradient pressure into the context encoder comes from the probe, and the remaining 30% comes from the dynamics model, which provides empirical evidence that such a probe is useful in expediting the training of the context encoder.

It may appear redundant to use the encoded context tokens in the model if the context encoder is also trained to predict the timestep, boundary conditions and context parameters. However, another interpretation is that the latent context embeddings allow far more expressivity of the fluid dynamics than what can simply be compressed into the values predicted by the probe. This means that the context encoder would be able to encode entirely new dynamics, which could not be expressed with only the set of values measured by the probe. From this perspective, the probe acts as an additional training signal, since the same abstract representations that would be necessary to predict the probe outputs would also be necessary for representing the relevant abstract dynamics that could generalise over unseen fluid systems.

3.3. Training Process

The dynamics model and context encoder probe were trained simultaneously on 20/24 of the generated meshes in the corpus of training data, with a cosine-annealing learning rate of 1×10^{-4} . A held out validation dataset of the trajectories from the remaining two meshes was used to observe model progress. The dynamics model was trained for four epochs of the training dataset, taking roughly 16 hours of wall clock time. The detail model was subsequently trained for a further 6 hours on the output of the frozen dynamics model, again over 4 epochs.

4. Results and Analysis

Proper evaluation of the model requires consideration of the model’s performance along three axes. Firstly, the model should be able to accurately predict future frames that appear generally inline with the ground truth data (generated by the numerical solver) without serious artefacts during autoregressive rollout. Secondly, the model output should respect physical constraints/invariants as far as possible – there shouldn’t be any sustained, unphysical drifts in energy or loss of detail. Finally, and most relevant to the contribution of this work, the model should correctly interpret the context it is fed, to generate frames that closely match the dynamics and boundary conditions of the context which it was given. The ultimate test of the success of this approach is to observe how well it can perform on unseen viscosity/PDE relationships, where the context encoder probe would not be able to contribute to its ability to accurately convey the relevant context to the dynamics model. However, in order to evaluate the third, the model must be sufficiently capable of solving PDEs close to the training distribution, in order for us to be able to attribute any failure of out of sample prediction to the context encoder setup.

4.1. Artefacts over Rollout

The best method for evaluating artefacts is simply by visually analysing the fluid states to check for any noticeable irregularities that appear over the course of a rollout. Despite modifying the dynamics model to predict the frame delta, this architecture appears to be robust against structural artefacts, just as observed in the results presented in the first report. Much of this can be attributed to the overlapping patch decoder patches, which provide a graceful way for neighbouring patches to be encouraged to learn to synchronise their outputs such that there are no artefacts. The final convolutional layer also gives the model the ability to counteract any persistent geometric biases in the output, by learning corrective patterns at the convolutional level and applying them across the entire domain.

- Compare against new type of viscosity/PDE relationship with new and reused context to see if the context can encode genuinely new stuff.

-

4.2. Physical Accuracy

The trained model is tested against various test cases, in order to evaluate the extent to which it is able to handle out of sample rollouts.

The results demonstrate success of the model in two key areas. Firstly, the model is able to auto-regressively predict fluid states, given relevant context

The results show a clear ability for the model to accurately evolve fluid states in an autoregressive manner, with minimal artefacting

4.3. Context as an Anchor against Autoregressive Drift

The context encoder also has the capacity to effectively counteract autoregressive drift in the fluid state. This is because the context is able to embed the macroscopic invariants of the system (i.e. the system Hamiltonian) as well as the fluid parameters and boundary conditions. Provided that the model is trained over a multi-step horizon via back-propagation through time (BPTT), the model can use the context tokens to correct any small drift that occurs during rollout.

This can be tested by training a probe to determine whether energy information is contained within the encoder tokens. If it is possible to obtain a reliable estimate of energy from this probe, then there is a reasonable probability that the model is relying on the context in order to correct for autoregressive drift.

4.4. Tent Weightings

Tent weightings actually are really useful because it allows dynamics to occur within patches due to the patches themselves increasing in size.

5. Comparison to Existing Models

As outlined in the brief review in the first report, and the introduction of this report, the use of machine learning in the field of CFD is a well established area of research. The architectural decisions taken for this project reflect a set of tradeoffs that are important to address.

5.1. In-Context Representation

Various different models represent the context in different ways. The ICON[23], VICON[2] and Zebra[20] approach is the ‘standard’ approach to in-context learning, which involves including sequential context frames in the input of the model. Hence, the model has access to all context frames at for every prediction step that the model performs. These models rely on observed trajectories (in the case of ICON, sets of input-output pairs) which demonstrate the operator that the model is trying to in-context learn.

However, as alluded to previously, a potential problem with this approach is the difficulty of attributing the observed behaviours to in-context learning from the provided context trajectories. In theory, it could very well be possible that these models rely on spurious relationships between observed frames and operator behaviour, such that when shown an out of distribution fluid state (i.e. a fluid state that looks like it would have been generated from dynamics A, but was in fact generated by dynamics B, which are accurately described by the context), the model infers incorrect dynamics and thus performs poorly.

The context that these models uses is never compressed/bottlenecked into a representation that can be clearly measured. However, since this project uses a context encoder, it is possible to directly measure the information stored in the context embeddings, which are by construction the only way that the model has access to the context data. For the model trained on the old training dataset, which included standard trajectories which used the same fluid parameters (so shared context) throughout, it was observed that the context contributed *almost nothing* to the output. The context tokens that the context encoder was trained to output were approximately zero, and did not vary with different context inputs.

This motivates the updated training data, with context switching along trajectories produced by the numerical solver for the training corpus. It also motivates the choice to include a training probe to add pressure for the context encoder to learn relevant information from the context frames. Once these changes were made, the context tokens were observed to encode useful information from the context frames.

This invites the question of whether these other in-context surrogate models suffer from the same problem. On the surface, this problem is not easily visible, if the validation data follow a trajectory which is drawn from the same distribution as the training dataset. However, if the model were ever applied in cases where the fluid state may look like it came from a different trajectory with a different context to the actual context specified, it is likely that these models would perform poorly (as observed in this project). Neither ICON, VICON nor Zebra address this issue explicitly nor implicitly through their chosen training regimes.

5.2. Modelling Uncertainty

CFD surrogate models differ substantially on how they approach the fact that there is uncertainty with how fluids behave. Many approaches, including Fourier neural operators (FNOs) such as Li et al. [10] and DeepONet (Lu et al. [13]) and foundation scale models such as Poseidon (Herde et al. [8]) and GPhyT (Wiesner et al. [22]) simply ignore the non-determinism, and treat it like a deterministic problem. This makes sense when we consider that we are training from a deterministic numerical solver. However, a benefit of machine learning is that it is probabilistic by nature, so it is the perfect candidate to learn a distribution over potential future fluid states.

The initial stochastic methods (Ravuri et al. [18]) used generative adversarial networks (GANs), similar to the model presented in the first report. The adversarial loss encourages the model to predict a distribution in order to fool the adversarial network. If the discriminator network is hypothetically infinitely good, then the only way that the generator model can fool the discriminator (and thus minimise its own loss) is by generating a output that is within the distribution of true potential outcomes. However, GAN models suffer from *mode collapse*, which is the when model tends to always predict the most likely point on distribution, because that is the point at which the discriminator finds it hardest to discriminate. Furthermore, GANs exhibit training instability, since both the generator and discriminator must roughly keep pace with each other. If the adversarial network becomes too powerful (which was frequently observed when training the first report model) then it provides extreme gradients to the generative model which cause it to destabilise.

End-to-end distribution matching models have also been investigated. Diffusion models, which iteratively denoise an starting image, have been tried [9, 19], although such models are very compute intensive both at training and inference time. These allow uncertainty seeded on the initial starting Gaussian, which is then progressively decoded in such a way that the target distribution is sampled. The flow matching model used in this project performs a similar function, while requiring significantly less compute.

6. Concluding Remarks

Machine learning is useful when we have systems which are *too complex to model* using numerical/deterministic computational methods, but where there are substantial efficiencies that can be achieved by the recognition of high level abstract patterns. For fluid dynamics specifically, there are such high level objects that tend to behave in .

However, CFD modelling is often performed precisely when it is new information that is sought, which is exactly the kind of setup that is likely to be out of distribution of the training data. For a CFD model to be useful in these cases, it needs to be able to generalise extremely well. The reality is that in the limit of this ability to generalise over any fluid, we would exactly replicate what a classical numerical solver can already give us.

The problem with a machine learning model is that machine they are essentially black boxes, which makes it very difficult to determine how the model has reached its prediction. For certain tasks, where the processes are highly complex and imprecise by definition, such as synthesising language, this is perfectly acceptable. However, for CFD modelling it is often critical to be able to understand the process by which the solver generated its result. Regardless of how accurate any machine learning model may be, for an out of sample prediction, there is no way to know what biases, errors or artefacts may exist in its output.

This issue is only compounded by the sheer scale of potential fluid permutations that exist. For other scientific applications such as protein folding, where the protein structures live on a

much lower dimensional manifold within a high dimensional space, machine learning methods can be extremely effective. However, for CFD modelling, our manifold is the set of points in phase space, which is truly massive. This means that, while it is possible to effectively learn a subset of dynamics, as shown in this report, in order to truly develop a fluid foundation model able to be universally applied like a numerical solver can would require such vast amounts of data. Even if such a model were to exist, it still only accurate to the level of precision that the data granularity allows.

References

- [1] Sahil Bhola and Karthik Duraisamy. Flow matching operators for residual-augmented probabilistic learning of partial differential equations, 2025.
- [2] Yadi Cao, Yuxuan Liu, Liu Yang, Rose Yu, Hayden Schaeffer, and Stanley Osher. VICON: Vision in-context operator networks for multi-physics fluid dynamics prediction. *arXiv preprint arXiv:2411.16063*, 2024.
- [3] Sipeng Chen, Junliang Liu, Hewei Tang, and Shibo Li. Multi-fidelity flow matching: Cascaded refinement of PDE solutions, 2026.
- [4] Yifan Chen, Mark Goldstein, Mengjian Hua, Michael S. Albergo, Nicholas M. Boffi, and Eric Vanden-Eijnden. Probabilistic forecasting with stochastic interpolants and föllmer processes. In *Proceedings of the 41st International Conference on Machine Learning (ICML)*, PMLR 235, 2024. doi: 10.48550/arXiv.2403.13724.
- [5] Yilong Dai, Yiming Sun, Yiheng Chen, Shengyu Chen, Xiaowei Jia, and Runlong Yu. FlowRefiner: Flow matching-based iterative refinement for 3D turbulent flow simulation, 2026.
- [6] Prafulla Dhariwal and Alexander Nichol. Diffusion models beat GANs on image synthesis. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2021. doi: 10.48550/arXiv.2105.05233.
- [7] Xiantao Fan, Deepak Akhare, and Jian-Xun Wang. Neural differentiable modeling with diffusion-based super-resolution for two-dimensional spatiotemporal turbulence. *Computer Methods in Applied Mechanics and Engineering*, 433:117478, 2025. doi: 10.1016/j.cma.2024.117478.
- [8] Maximilian Herde, Bogdan Raonić, Tobias Rohner, Roger Käppeli, Roberto Molinaro, Emmanuel de Bézenac, and Siddhartha Mishra. Poseidon: Efficient foundation models for PDEs. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.
- [9] Georg Kohl, Li-Wei Chen, and Nils Thuerey. Benchmarking autoregressive conditional diffusion models for turbulent flow simulation. *Neural Networks*, 2026. doi: 10.1016/j.neunet.2026.108641.
- [10] Zongyi Li, Nikola Kovachki, Kamyar Azizzadenesheli, Burigede Liu, Kaushik Bhattacharya, Andrew Stuart, and Anima Anandkumar. Fourier neural operator for parametric partial differential equations. In *International Conference on Learning Representations (ICLR)*, 2021.

- [11] Marten Lienen, David Lüdke, Jan Hansen-Palmus, and Stephan Günnemann. From zero to turbulence: Generative modeling for 3D flow simulation. In *International Conference on Learning Representations (ICLR)*, 2024. doi: 10.48550/arXiv.2306.01776.
- [12] Yaron Lipman, Ricky T. Q. Chen, Heli Ben-Hamu, Maximilian Nickel, and Matt Le. Flow matching for generative modeling. In *International Conference on Learning Representations (ICLR)*, 2023. doi: 10.48550/arXiv.2210.02747.
- [13] Lu Lu, Pengzhan Jin, Guofei Pang, Zhongqiang Zhang, and George Em Karniadakis. Learning nonlinear operators via DeepONet based on the universal approximation theorem of operators. *Nature Machine Intelligence*, 3:218–229, 2021. doi: 10.1038/s42256-021-00302-5.
- [14] Morteza Mardani, Noah Brenowitz, Yair Cohen, Jaideep Pathak, Chieh-Yu Chen, Cheng-Chin Liu, Arash Vahdat, Mohammad Amin Nabian, Tao Ge, Akshay Subramaniam, Karthik Kashinath, Jan Kautz, and Mike Pritchard. Residual corrective diffusion modeling for km-scale atmospheric downscaling. *Communications Earth & Environment*, 6(124), 2025. doi: 10.1038/s43247-025-02042-5.
- [15] Jaideep Pathak, Yair Cohen, Piyush Garg, Peter Harrington, Noah Brenowitz, Dale Durran, Morteza Mardani, Arash Vahdat, Shaoming Xu, Karthik Kashinath, and Michael Pritchard. Kilometer-scale convection-allowing model emulation using generative diffusion modeling. *Science Advances*, 12:eadv0423, 2026. doi: 10.1126/sciadv.adv0423.
- [16] Ethan Perez, Florian Strub, Harm de Vries, Vincent Dumoulin, and Aaron Courville. FiLM: Visual reasoning with a general conditioning layer. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 32, 2018. doi: 10.1609/aaai.v32i1.11671.
- [17] Ilan Price, Alvaro Sanchez-Gonzalez, Ferran Alet, Tom R. Andersson, Andrew El-Kadi, Dominic Masters, Timo Ewalds, Jacklynn Stott, Shakir Mohamed, Peter Battaglia, Remi Lam, and Matthew Willson. Probabilistic weather forecasting with machine learning. *Nature*, 637:84–90, 2025. doi: 10.1038/s41586-024-08252-9.
- [18] Suman Ravuri, Karel Lenc, Matthew Willson, Dmitry Kangin, Remi Lam, Piotr Mirowski, Megan Fitzsimons, Maria Athanassiadou, Sheleem Kashem, Sam Madge, Rachel Prudden, Amol Mandhane, Aidan Clark, Andrew Brock, Karen Simonyan, Raia Hadsell, Niall Robinson, Ellen Clancy, Alberto Arribas, and Shakir Mohamed. Skilful precipitation nowcasting using deep generative models of radar. *Nature*, 597:672–677, 2021. doi: 10.1038/s41586-021-03854-z.
- [19] Salva Rühling Cachay, Bo Zhao, Hailey James, and Rose Yu. DYffusion: A dynamics-informed diffusion model for spatiotemporal forecasting. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023. doi: 10.48550/arXiv.2306.01984.
- [20] Louis Serrano, Armand Kassaï Koupaï, Thomas Wang, Pierre Erbacher, and Patrick Gallinari. Zebra: In-context generative pretraining for solving parametric PDEs. In *Proceedings of the 42nd International Conference on Machine Learning (ICML)*, PMLR 267, 2025. doi: 10.48550/arXiv.2410.03437.

- [21] Jianlin Su, Murtadha Ahmed, Yu Lu, Shengfeng Pan, Wen Bo, and Yunfeng Liu. Roformer: Enhanced transformer with rotary position embedding. *Neurocomputing*, 568: 127063, 2024. doi: 10.1016/j.neucom.2023.127063.
- [22] Florian Wiesner, Matthias Wessling, and Stephen Baek. Towards a physics foundation model. *arXiv preprint arXiv:2509.13805*, 2025.
- [23] Liu Yang, Siting Liu, Tingwei Meng, and Stanley J. Osher. In-context operator learning with data prompts for differential equation problems. *Proceedings of the National Academy of Sciences*, 120(39):e2310142120, 2023. doi: 10.1073/pnas.2310142120.
- [24] Max Zhu. Pytorch finite volume method solver. 2026. URL https://github.com/Maxzhu123/fvm_solver.

Appendix A. The Transport-Residual Architecture

While many different architectures were considered, one of the most interesting, and worth including, is the ‘transport-residual’ architecture, which was constructed of two stages, a transport/advection stage, and a residual detail stage. While this architecture successfully learned as it should, it could not compare with the more expressive method described in the main body of this report.

Appendix A.1. Transport Model

The advection stage of the model is designed as an autoencoder, which encodes/decodes a latent space describing the transport map to advect one fluid state to another. The autoencoder uses an image transformer, with the encoded latents appended as query tokens. This model is trained by taking many ordered pairs of sequential fluid states and training the autoencoder to produce a latent that allows the decoder to decode a transport map that produces an output close to that of the input.

The core challenge when training the model to output a transport map, is that in general, there is no clear gradient down which to follow to converge. This is because, unlike when predicting actual cell values, if our transport map vectors are close but not quite right, the loss may not be close to the loss yielded by the correct map. In other words, the model loss is not monotonic in proximity to the target transport map, which means that gradient descent along the slope of steepest descent with respect to the model parameters does not generally converge to a desirable transport map.

In order to rectify this, a special training curriculum is used to encourage the model to initially produce a roughly uniform transport map across the domain. The reconstruction error is multiplied by a measure of the local complexity of the *transport map* (not of the field it acts on), evaluated over non-overlapping $w \times w$ windows $\mathcal{W}$:

$$L_{\text{trans}} = \frac{1}{|\mathcal{W}|} \sum_{w \in \mathcal{W}} \bar{e}_w \left(1 + \lambda_d \frac{\text{Var}_w(\mathbf{d})}{d_{\text{max}}^2} + \lambda_g \frac{|\bar{g} - 1|_w}{g_{\text{range}}} \right), \quad (\text{A.1})$$

where the per-pixel error and its window mean are

$$e_i = \|\hat{x}_i - x_i\|_2^2 + \lambda_1 \|\hat{x}_i - x_i\|_1, \quad \bar{e}_w = \frac{1}{|w|} \sum_{i \in w} e_i, \quad (\text{A.2})$$

and the map complexity is the within-window variance of the displacement field $\mathbf{d} = (d_x, d_y)$, averaged over its two components,

$$\text{Var}_w(\mathbf{d}) = \frac{1}{2} \sum_{c \in \{x, y\}} (\overline{d_{c,w}^2} - \bar{d}_{c,w}^2), \quad \overline{|g - 1|}_w = \frac{1}{|w|} \sum_{i \in w} |g_i - 1|. \quad (\text{A.3})$$

The normalisations by the per-step displacement bound $d_{\max}$ and the gain range g_{range} render λ_d and λ_g dimensionless.

The modulation is expressed as $(1 + S)$ rather than simply $\bar{e}_w S$, since this would admit a degenerate minimiser at $S \rightarrow 0$ with arbitrary reconstruction error; in the form of Eq. (A.1) complexity must pay for itself through a proportional reduction in that window’s own error.

The idea is that while the error is high, any attempt to predict a complex, uneven field should be penalised, since the gradient tells us very little about in which direction our parameters should move. In this case, the model is encouraged to predict a more uniform field as a way to decrease the loss. However, when predicting a uniform field, the gradient signal now becomes more useful, since it is averaged over a larger area. Once the model has approximately minimised the loss at a certain level of uniformity, the only way it can decrease the loss further is to simultaneously increase the variance but also predict more accurately. In order to encourage this, the λ_d parameter is slowly reduced as the training run continues, in order to encourage the model to penalise variance less and admit more complexity to the transport map.

Appendix A.2. Residual Detail Model

The detail model is a generative model which takes the output of the transport stage and adds detail which cannot effectively be captured by the transport map alone. This detail includes chaotic features for which the prediction cannot be deterministic, but also deterministic features which simply cannot be effectively represented by transport alone.